M4GB integration and kernel speed-up (2026-11-10)

M4GB integration and kernel speed-up (2026-11-10)

Study directory for the M4GB work: the external-process backend (`src/cryptanalysis/m4gb_oracle.rs`, `m4gb-f2` in the `SystemSolver` registry), the pinned build (`scripts/m4gb/build.sh`, `third_party/m4gb-patches/`) and the A/B measurement of the two patches. Assessment and plan: `docs/ic/M4GB_INTEGRATION_ASSESSMENT_20261110.md`.

Everything measured here is a **local development check on the session container** (4 vCPU Intel Xeon 2.10 GHz, AVX-512, 16 GiB, shared with a Rust build running concurrently), not an isolab run: wall times are `[pcc:unverified]` and the lab measurement is the open thread. Benchmark record: `benchmarks/records/20261110T0551Z-m4gb-arith-kernels-ab.json`.

What was changed in M4GB

Upstream `cr-marcstevens/m4gb` at `01860a52` (2026-09-25). Profile of the upstream solver on a random dense `GF(31)` system (`n = 15`, `m = 30`, one thread, gprof): about 90 % of samples in the dense-row kernels (`gb::add_to` and the polymatrix row reduction that inlines it). Upstream computes `l[i] = l[i] + c·r[i] mod p` with one three-dimensional table gather per coefficient (`addmul[c][r][l]`, `p ≤ 64`) or a two-dimensional one (`p ≤ 256`).

- **Patch 0001** (`lib/gf_p_simple.hpp`): the vector kernels `add_to`, `mul_to`, `substract_to` use plain integer arithmetic with the compile-time modulus. For $p\cdot(p-1)^2 < 2^{16}$ the reduction is one 16-bit multiply-high ($q = (t\cdot\lceil 2^{16}/p\rceil) \gg 16$, exact on $0 \le t \le p(p-1)$); otherwise the compiler's constant-modulus sequence. Kernel microbenchmark on this host, $p = 31$, 4,096 coefficients: table 0.81 ns per coefficient, `%` 0.27, magic 0.12 (6.8× over the table) `[pcc:unverified]`.
- **Patch 0002** (`lib/parser.hpp`, `src/m4gb.hpp`): over `GF(2)` the field equations are added only for the variables the input declares or uses. A binary compiled for `MAXVARS = 128` otherwise carries every unused variable through the computation; on the 35-variable Boolean systems below that doubled the time, and on the `n = 30` random system the 64-variable binary took 3.7× the 30-variable binary's time before the patch.

Both patches are switchable (`-DM4GB_ARITH_KERNELS=0`, `-DM4GB_FIELDEQ_DECLARED_ONLY=0`), which is how the baseline arm is built from the same tree. Upstream's `make check` (10 tests) passes on the patched tree.

Where the GF(2) time goes (patched solver, `root1`, 35 variables, one thread, gprof)

`m4gb::update()` (Gebauer–Möller critical-pair update) 42 % of samples over 3,574 calls; `polymatrix::get_u_g` 10 %; the dense XOR kernel `add_to` 10 %; the rest is spread over symbol-aliased small functions (gprof attributes 15 % to `solver_base_t::clear()` with 42.8 M calls and 11 % to `frame_dummy`, which is symbol confusion, not a measurement). The dense kernel is therefore not the lever over `GF(2)`; the pair update is, and it is algorithmic work (the second loop builds a `std::map` over the whole basis per update, the first scans every pending pair with an integer-monomial divisibility test). Left as the next target `[pcc:unverified]`.

Protocol

- Arms: `base` = pinned upstream with both switches off; `new` = both patches on. Same compiler (`g++ 14`, `-O3 -march=native`), same tree, binaries checked to differ by hash.
- Inputs: random dense quadratic `GF(31)` systems (`m = 2n`, `n = 12, 14, 15, 16`, planted root, seeds 1, 3, 1, 2; M4GB's own generator); three of the `GF(65521)` systems of `research/gb_cas_comparison_20261011`; the eight `n = 17` Boolean decomposition root systems of the same comparison (35 variables, converted from the msolve inputs by `tools/ms2m4gb.py`-style conversion, field equations dropped because M4GB adds them); upstream's Weil-descent regression system `wd_n16`; a random dense `GF(2)` system `n = 30`, `m = 60`.
- One thread (`--nrthreads 1`), `ulimit -v` 6 GB, timeout per run, three repeats (two for the slowest cells, one for the slowest), arms interleaved within each repeat (odd repeats base first, even repeats new first).
- Correctness: output files hashed per run; the two arms must produce the same basis (for `GF(2)` with a wide binary, modulo the field equations of undeclared variables that upstream prints as `^2 +`); planted roots compared with the generator's `.ans`.
- Pre-stated expectations (written before the valid run; a first run was discarded because its "baseline" binaries turned out identical to the patched ones by hash, see the session log): patch 0001 gives at least 2× on the `GF(31)` systems at `n ≥ 14` and less on `GF(65521)` (16-bit storage, no magic path); patch 0002 gives about 2× on the 35-variable systems under the 128-variable binary and nothing under the 35-variable binary; no change on `GF(2)` under an exact-width binary.

Results

(filled from `results.tsv` by `ab_report.py`; see below)

input	binary	n	base median s (min–max)	patched median s (min–max)	speedup (median)
g31_n12	n20_gf31	3	0.539 (0.503–0.576)	0.138 (0.136–0.151)	3.91×
g31_n14	n20_gf31	3	1.122 (1.106–1.185)	0.294 (0.275–0.300)	3.82×
g31_n15	n20_gf31	3	1.609 (1.504–1.658)	0.390 (0.375–0.406)	4.13×
g31_n16	n20_gf31	3	7.201 (6.944–7.537)	2.116 (2.046–2.201)	3.40×
overdet_n12_m24_p65521	n12_gf65521	3	0.140 (0.109–0.148)	0.086 (0.067–0.092)	1.63×
overdet_n10_m15_p65521	n12_gf65521	3	0.091 (0.079–0.105)	0.048 (0.046–0.051)	1.90×
quad_n9_p65521	n12_gf65521	3	7.445 (6.922–7.471)	3.749 (3.578–3.974)	1.99×
root1	n35_gf2	3	8.291 (8.176–8.477)	8.236 (7.759–8.624)	1.01×
root2	n35_gf2	3	2.025 (1.870–2.035)	1.922 (1.892–1.952)	1.05×
root3	n35_gf2	3	4.538 (4.482–4.833)	4.976 (4.875–5.187)	0.91×
root4	n35_gf2	3	7.685 (7.622–7.709)	7.542 (7.329–7.753)	1.02×
root5	n35_gf2	3	8.802 (8.746–9.116)	8.822 (8.729–8.941)	1.00×
root6	n35_gf2	3	2.064 (1.952–2.066)	2.091 (2.042–2.110)	0.99×
root7	n35_gf2	3	1.821 (1.767–2.446)	1.977 (1.751–2.373)	0.92×
root8	n35_gf2	3	8.242 (7.774–8.466)	8.330 (8.024–8.380)	0.99×

input	binary	n	base median s (min–max)	patched median s (min–max)	speedup (median)
root1	n128_gf2	3	15.303 (14.624–15.711)	13.831 (12.453–14.074)	1.11×
root3	n128_gf2	3	15.428 (15.152–15.582)	14.989 (14.781–15.054)	1.03×
wd_n16	n64_gf2	3	1.183 (1.021–1.183)	0.941 (0.892–1.079)	1.26×
g2_n30	n30_gf2	2	128.517 (126.699–130.335)	123.691 (117.392–129.990)	1.04×
g2_n30	n64_gf2	1	27.899 (27.899–27.899)	27.559 (27.559–27.559)	1.01×